Barcode Utility (barcodeutil.exe)

Operation, Queue Capacity, File Detection, and Concurrent User Load

Purpose: explain how the current application works, how its queue behaves, how files are detected, and what to consider when many users submit labels at once.

> **Scope and assumptions**
> This guide describes the current design discussed for barcodeutil: a hidden .NET application that watches a local server folder for `.barcode` files, moves ready files to a processing folder, reads the printer UNC path from line 2, removes the first two header lines from the print payload, optionally replaces `***COMPANYLOGO***`, and copies the resulting temporary file to the printer queue. Configuration holds the watch folder, destination folder, and logo file. The application uses a bounded in-memory queue with capacity 10,000, a FileSystemWatcher plus a 5-second polling scan, and one worker that processes jobs sequentially.

Important: the queue capacity and other behavior described below are based on the current source previously shared. Verify against the exact source/build deployed on the server before treating this document as an audited production specification.

1. How can we increase the queue size?

The current queue is a bounded in-memory `BlockingCollection<string>` configured with `QueueCapacity = 10000`. Each queued item is a file path, not the label contents.

To raise it, change the constant in `Program.cs`:

```
private const int QueueCapacity = 20000;
```

The collection is created using that constant, so changing it and rebuilding/publishing increases the queue limit. Choose a number based on measured arrival rate and how quickly the printer can consume jobs. A larger queue buffers bursts; it does not make printing faster or increase printer throughput.

Current full-queue behavior: `TryAdd` is non-blocking. If the queue is full, the program removes the path from the deduplication set and logs `[Queue Full]`. Polling may discover the file again later, but this is not a guarantee under all conditions. Monitor this log entry and test recovery before production.

Recommendation: do not increase the limit blindly. Measure peak submissions, print duration, queue-full events, memory, and disk usage. If bursts routinely exceed 10,000 files, increase capacity (for example, 20,000) and consider a durable disk-backed queue if jobs must survive process/server restarts.

2. Does the program use kernel notifications or polling?

It uses both:

FileSystemWatcher: uses Windows file-system change notifications to signal events such as a new file or rename. This is the fast path; it avoids repeatedly scanning the folder for every new arrival.

Polling: every 5 seconds, the application enumerates the watch folder and enqueues matching `.barcode` files. This is a safety net for missed notifications, watcher-buffer overflow, or startup timing issues.

The watcher is configured to watch file-name changes and the application waits for the file size and last-write timestamp to stabilize, then attempts an exclusive read open before moving it. This is intended to avoid processing a file while a user is still writing it.

Important limitation: neither FileSystemWatcher nor a size-stability check proves that an upstream application has finished writing in every possible case. The current design is best-effort detection with retries and polling. A producer-side temporary-file-and-rename protocol would be stronger, but is not assumed here because users' generating applications cannot be changed.

3. What can go wrong with hundreds of users submitting at once?

Risk	What it means / response
Queue saturation	A burst can exceed 10,000 queued paths. The current non-blocking enqueue logs [Queue Full]. Increase capacity only after measuring; monitor the log and confirm polling later rediscovers files.
Printer throughput bottleneck	One worker sends jobs sequentially. If labels arrive faster than the printer can accept them, backlog grows even when detection is healthy. A bigger queue only absorbs the burst.
Files still being written	A file event can occur before the network copy completes. Stability and exclusive-open checks reduce the risk, but a long or paused copy can hit the readiness timeout and be retried by later polling.
Duplicate events	Watcher and polling can find the same file. The queued-path dictionary suppresses duplicate queue entries while a path is already queued. The path is removed from that set after processing; stable unique filenames are important.
Name collisions	Two users writing the same filename can collide in the shared folder or destination folder. Filenames should include a unique user/job identifier, ideally with a unique suffix.
SMB/share permissions	Users need write access to the input share; the application account needs local folder access and permission to write to every printer share referenced on line 2.
Printer unavailable or slow	Printing is retried three times with a 2-second delay between attempts. On failure, the original file remains in the destination folder and the failure is logged. A backlog can grow if the printer is unavailable.
Memory queue is not durable	Queued paths exist in memory. A process/server restart loses the queue state. Files still in the watch folder should be found by the startup scan; files already moved to the destination folder are subject to startup cleanup and may be deleted.
Disk/log pressure	Large bursts consume disk space in input/destination folders and produce more log entries. Ensure free disk space and log/archive retention are monitored.
Many printer destinations	The printer is selected per file from line 2. Validate that the line is a legitimate UNC printer path and that the service identity has access. Consider an allow-list if input files are not fully trusted.

Concurrency note: hundreds of users can create files concurrently without creating hundreds of print workers. File detection and queuing can accept many arrivals, but actual printing is serialized by one worker. This protects the printer from overlapping writes, while also making printer throughput the main capacity constraint.

4. Step-by-step: how the program works

1. Start

The executable starts in the background and writes messages to `barcodeutil.log` beside the executable.

2. Read configuration

It loads `barcodeutil.config`, including `WatchFolder`, `DestinationFolder`, and `LogoFile`. Missing required values cause startup to fail.

3. Prepare folders and logo

It creates the watch and destination folders if needed and loads the logo content using Latin-1 encoding. A missing logo file causes startup to fail.

4. Clean the destination folder

At startup, it deletes existing files from the destination folder. This is a deliberate cleanup behavior, but means files left there after a failed print can be lost on restart.

5. Start detection

It starts FileSystemWatcher before the initial scan, reducing the chance of missing files arriving during startup.

6. Detect .barcode files

Created/renamed events enqueue paths. A polling scan also runs every 5 seconds and discovers matching files.

7. Deduplicate and queue

The queued-path dictionary prevents the same path being queued repeatedly at the same time. The bounded queue holds up to 10,000 pending paths.

8. Wait until the file is ready

The worker checks that size and last-write time are stable and tries to open the file exclusively. It retries for a limited time if the file appears to still be in use.

9. Move to processing folder

The worker moves the input file from the watch folder to the destination folder, retrying transient move failures.

10. Read the printer path

It reads line 1 and ignores it, reads line 2 as the printer UNC path (for example, `\\PrintingServer\BarcodePrintingQueue` then reads the remainder as the printer payload. File beginning 4 lines example:

```
\\PrintingServer\BarcodePrintingQueue
DOWNLOAD F,"WIPLABEL.BAS"
OPEN "WIPLABEL.DAT" FOR INPUT AS #1
```

11. Prepare print payload

The first two lines are omitted from the payload. If the remaining data contains `***COMPANYLOGO***`, the marker is replaced with the loaded logo content.

12. Create a temporary print file

The payload is written to a uniquely named temporary file. The original .barcode in the destination folder is not rewritten.

13. Send to printer

The application copies the temporary file to the UNC printer queue from line 2. It tries up to three times, waiting two seconds between failed attempts.

14. Record outcome

Success and failure are logged. On failure, the original input file remains in the destination folder for inspection until cleanup removes it.

15. Poll and clean daily

Every 5 seconds the polling loop also checks whether the daily cleanup is due. After 3:00 AM, once per day, it deletes destination files whose last-write time is before today's date, skipping files currently marked as processing.

5. Current operating parameters

Setting	Current behavior
Input extension	.barcode
Detection	FileSystemWatcher plus 5-second polling
Queue capacity	10,000 pending file paths; in memory
Workers	1 sequential worker
Readiness wait	Up to 20 attempts, with delays; approximately 14 seconds in the common timeout path
Move retries	Up to 10 attempts
Print retries	3 attempts, 2 seconds between failures
Print payload	All content after the first two lines, with optional logo-marker replacement
Log rollover	At approximately 1 MB, current log is renamed to a timestamped archive
Startup cleanup	Deletes files already in DestinationFolder
Daily cleanup	After 3 AM, removes destination files last modified before today

6. Recommended production checks

• Track [Queue Full], [File Not Ready], [Move Failed], and [PRINT FAILED] log entries.

• Load-test with representative file sizes and a test printer before allowing a large user population.

• Confirm the account running the application can write to all printer UNC paths that may appear on line 2.

• Ensure filenames are unique and users cannot overwrite another user's in-progress file.

• Monitor the input and destination folder sizes and free disk space.

• Decide whether failed files must survive application/server restarts. If yes, revise startup cleanup and use a durable recovery strategy.

• If jobs must never be lost, consider a disk-backed job queue, explicit job states, and a way to reconcile ambiguous print outcomes. Printing can be ambiguous if the share accepts a file but an error occurs before success is recorded.

• For service deployment, test permissions using the actual service account, especially access to remote printer shares.

7. Bottom line

The current program is designed to absorb bursts of file arrivals and send labels to the printer sequentially. FileSystemWatcher gives fast notification; 5-second polling is the recovery safety net. Raising QueueCapacity provides more burst buffer but cannot increase printer speed. With hundreds of simultaneous submitters, the key controls are unique filenames, permissions, free disk space, monitoring queue-full and print-failure logs, and testing the actual printer throughput.